

Unit 2

RELATIONAL MODEL AND SQL

The **Relational Data Model (RDM)** is the foundation of relational databases and represents data using **tables (relations)**. It was introduced by **E.F. Codd** in 1970 and is based on **mathematical set theory** and **predicate logic**. Here's a breakdown of its core components:

♦ 1. Schema

- A **schema** is the **structure** or **blueprint** of a database.
- It defines the **name of the relation (table)**, and its **attributes (columns)** and **data types**.
- Example:

```
Student(SID: int, Name: string, Age: int, Dept: string)
```

- o Here, `Student` is a relation.
 - o `SID`, `Name`, `Age`, `Dept` are attributes.
-

♦ 2. Tuples

- A **tuple** is a **single row** in a relation.
- It represents a **single record or instance** of the entity.
- Example:

```
(101, 'Alice', 20, 'CSE')
```

This is a tuple in the `Student` relation.

♦ 3. Domains

- A **domain** is the **set of permissible values** for an attribute.
 - Each attribute in a relation is defined over a domain.
 - Example:
 - o Domain of `Age`: {18, 19, 20, ..., 60}
 - o Domain of `Name`: All strings of letters up to 50 characters
-

♦ 4. Integrity Rules

These are **constraints** to maintain **accuracy** and **consistency** of data:

✓ a. Entity Integrity Rule

- **Primary Key** must be **unique** and **not null**.
- Ensures each tuple can be uniquely identified.
- Example:

```
Student(SID PRIMARY KEY, Name, Age, Dept)
```

✓ b. Referential Integrity Rule

- Ensures **foreign key** values must match a **primary key** in another relation or be **null**.
- Maintains valid relationships between tables.
- Example:

```
Enroll(SID FOREIGN KEY REFERENCES Student(SID), CourseID)
```

Relational algebra: selection, projection, joins, set operations

Relational algebra is a procedural query language for the relational data model, forming the theoretical foundation for database query languages like SQL. It uses a set of operations to manipulate relations (tables). Key operations include:

- **Selection (σ):**
 - o Filters rows (tuples) from a relation based on a specified condition (predicate).
 - o Syntax:

$$\sigma_{\text{condition}}(R)$$

, where R is the relation and `condition` is a boolean expression.

- o Example:

$$\sigma_{\text{salary} > 50000}(\text{Employees})$$

would select all employees with a salary greater than 50,000.

- **Projection (π):**
 - o Selects specific columns (attributes) from a relation, effectively removing other columns.
 - o Syntax:

$$\pi_{A_1, A_2, \dots, A_n}(R)$$

, where R is the relation and A1, A2, ..., An are the desired attributes.

- o Example:

$\pi_{name, department}(Employees)$

would retrieve only the name and department columns from the Employees relation.

- **Joins ($\bowtie$):**

- o Combines rows from two or more relations based on a common attribute or a specified condition.
- o **Natural Join ($\bowtie$):** Combines relations based on common attributes with the same name and domain, automatically eliminating duplicate columns.
- o **Theta Join ($\bowtie_{\theta}$):** Combines relations based on a general

condition θ

- o **Equijoin ($\bowtie_{=}$):** A specific type of Theta Join where the condition is an equality comparison.
- o **Outer Joins (LEFT, RIGHT, FULL):** Retain all tuples from one or both relations even if there's no match in the other relation, padding with nulls where necessary.

- **Set Operations:**

- o These operations are applied to relations that are union-compatible (have the same number of attributes and corresponding attributes have the same domains).
- o **Union ($\cup$):** Combines all unique rows from two relations.
- o **Intersection ($\cap$):** Returns rows common to both relations.
- o **Set Difference ($-$):** Returns rows present in the first relation but not in the second.
- o **Cartesian Product ($\times$):** Combines every row from the first relation with every row from the second relation, resulting in all possible combinations. This is often a preliminary step for other operations like joins.

Structured Query Language (SQL)

Structured Query Language (SQL) is a domain-specific language used for managing and manipulating data in relational database management systems (RDBMS). It serves as the standard language for interacting with databases like MySQL, Oracle, SQL Server, PostgreSQL, and others.

SQL include:

Data Definition Language (DDL):

This component of SQL is used to define and manage the structure of a database. Commands like CREATE TABLE, ALTER TABLE, and DROP TABLE fall under DDL.

Data Manipulation Language (DML):

DML commands are used for managing data within the database. Examples include INSERT (to add data), UPDATE (to modify data), and DELETE (to remove data).

Data Query Language (DQL):

The SELECT statement is the primary component of DQL and is used to retrieve data from the database based on specified criteria.

Data Control Language (DCL):

DCL commands are used for managing user permissions and access control within the database, such as GRANT and REVOKE.

Transaction Control Language (TCL):

TCL commands manage transactions within the database, ensuring data integrity. Examples include COMMIT and ROLLBACK.

SQL is a declarative language, meaning users specify what data they want to retrieve or manipulate, rather than how to achieve it. This abstraction simplifies database interactions and allows the database management system to optimize the execution of queries.

Data Definition Language (DDL)

DDL, or Data Definition Language, comprises SQL commands used to define, modify, and delete database objects and their structures. The primary DDL commands are CREATE, ALTER, and DROP.

CREATE: This command is used to build new database objects. It establishes the initial structure of objects like:

Tables: Defines columns, data types, and constraints.

Databases: Creates a new database instance.

Indexes: Creates indexes for faster data retrieval.

Views: Creates virtual tables based on the result set of a SQL query.

Example:

Code

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50)  
);
```

ALTER: This command is used to modify the structure of existing database objects. It can be used to:

Add, modify, or drop columns in a table.

Add or remove constraints (e.g., PRIMARY KEY, FOREIGN KEY).

Rename tables or columns.

Example:

Code

```
ALTER TABLE Students  
ADD COLUMN Email VARCHAR(100);
```

DROP: This command is used to remove database objects entirely. When an object is dropped, it and its associated data are permanently deleted.

Tables: Deletes a table and all its data.

Databases: Deletes an entire database.

Indexes, Views, etc.: Deletes the specified object.

Example:

Code

```
DROP TABLE Students;
```

DDL commands are distinct from Data Manipulation Language (DML) commands (like INSERT, UPDATE, DELETE), which operate on the data within the defined structures rather than the structures themselves. DDL commands are typically auto-committed, meaning changes are permanently saved upon execution.

DML (INSERT, UPDATE, DELETE)

DML stands for Data Manipulation Language, and its commands are used to manage and modify data within a database. The core DML commands for altering data are INSERT, UPDATE, and DELETE.

INSERT: This command adds new rows (records) to a table.

Purpose: To introduce new data into a database table.

Example:

Code

```
INSERT INTO Employees (EmployeeID, FirstName, LastName)
VALUES (101, 'John', 'Doe');
```

UPDATE: This command modifies existing data within one or more rows of a table.

Purpose: To change the values of specific columns in existing records.

Example:

Code

```
UPDATE Employees
SET LastName = 'Smith'
WHERE EmployeeID = 101;
```

DELETE: This command removes one or more rows from a table.

Purpose: To remove unwanted or obsolete records from a table.

Example:

Code

```
DELETE FROM Employees
WHERE EmployeeID = 101;
```

These DML commands are fundamental for maintaining and evolving the data stored in relational databases, allowing for dynamic data management based on application

requirements. Unlike DDL (Data Definition Language) commands, DML commands are typically not auto-committed and can often be rolled back to a previous state if necessary.

DCL, TCL (COMMIT, ROLLBACK, GRANT)

DCL and TCL in SQL: Managing access and transactions

In SQL, Data Control Language (DCL) and Transaction Control Language (TCL) are two important categories of commands used for database management.

DCL (Data Control Language)

DCL commands manage access and permissions within a database. They are essential for database security and controlling what users can do with the data.

GRANT

The GRANT command is used to assign specific permissions to users, allowing them to perform actions like selecting, inserting, updating, or deleting data in database objects (tables, views, etc.).

Example:

sql

```
GRANT SELECT, INSERT ON employees TO user1;
```

Use code with caution.

This grants SELECT and INSERT permissions on the employees table to user1.

REVOKE

The REVOKE command is used to remove permissions previously granted to users.

Example:

sql

```
REVOKE INSERT ON employees FROM user1;
```

Use code with caution.

This revokes the INSERT privilege on the employees table from user1.

TCL (Transaction Control Language)

TCL commands manage database transactions, ensuring data integrity and consistency.

COMMIT

The COMMIT command permanently saves all changes made during the current transaction to the database.

Example:

sql

```
BEGIN TRANSACTION;
```

```
UPDATE products SET stock = stock - 10 WHERE product_id = 1;
```

```
COMMIT;
```

This updates the stock for product_id = 1 and then uses COMMIT to make the change permanent.

ROLLBACK

The ROLLBACK command is used to undo changes made during the current transaction that have not yet been saved using COMMIT. It reverts the database to its state before the transaction began or to a specified SAVEPOINT.

Example:

sql

```
BEGIN TRANSACTION;
```

```
DELETE FROM orders WHERE order_id = 100;
```

```
ROLLBACK;
```

This deletes an order but the ROLLBACK command undoes the deletion because the changes were not committed.

SAVEPOINT

The SAVEPOINT command allows you to set checkpoints within a transaction. This enables you to roll back to a specific point without undoing the entire transaction.

Example:

sql

```
BEGIN TRANSACTION;
```

```
UPDATE employees SET salary = salary * 1.05 WHERE department = 'HR';
```

```
SAVEPOINT salary_update;
```

```
DELETE FROM employees WHERE department = 'HR';
```

```
ROLLBACK TO salary_update;
```


COMMIT;

In this example, the salary updates are saved, but the deletions are undone by rolling back to the savepoint before the deletions were made.